Shape vs. Appearance: Comparing DINOv2 and YOLOv8 on Original and Hollow Contour Images

Abstract

We investigate how well vision models can classify images using only shape outlines (Canny edge contours) compared to full-color originals. We compare two architectures—DINOv2 (a self-supervised Vision Transformer) and YOLOv8n-cls (a lightweight CNN classifier)—on two datasets: CIFAR-10 (10-class object recognition) and an Overwatch character recognition dataset (14-class game screenshots). On CIFAR-10, both models achieve ∼96% on originals, but diverge on contours: YOLOv8n retains 70.4% while DINOv2 drops to 51.6%. On Overwatch, YOLOv8n reaches 65.8% on originals and 38.1% on contours, while DINOv2 achieves 56.5% and 13.4% respectively. Across both datasets, fine-tuned CNNs consistently leverage shape information more effectively than frozen self-supervised features.

1 Introduction

A long-standing question in visual recognition is the relative importance of shape versus texture in how neural networks classify objects [1]. Human perception is strongly shape-biased, yet convolutional neural networks have been shown to rely more on local texture cues. Self-supervised Vision Transformers such as DINO [2] have been reported to develop more shape-aware representations, attending to object boundaries even without explicit supervision.

In this study, we test this claim empirically by removing all texture and color information from images, leaving only Canny edge contours (hollow outlines). We evaluate on two datasets of different nature:

- **CIFAR-10**: A standard 10-class object recognition benchmark (50k train, 10k test; native 32×32).
- **Overwatch Frames**: A custom 14-class dataset of game character screenshots (1,309 train, 336 test; native 3420×2138), containing both normal and occluded views.

We compare two contrasting architectures:

- **DINOv2** (ViT-S/14): A frozen self-supervised Vision Transformer with a linear classification head trained on the target data.
- **YOLOv8n-cls**: A lightweight convolutional classifier, fully fine-tuned from pretrained ImageNet weights.

2 Method

2.1 Dataset Preparation

For both datasets, we generate contour versions by: (1) converting to grayscale, (2) applying Canny edge detection (thresholds: 100, 200), and (3) saving the binary edge map as a 3-channel PNG. All images are resized to 224×224 at training/inference time.

For the Overwatch dataset, images from both the `normal/` and `occluded/` subfolders are merged per character, then split 80/20 into train/test sets.

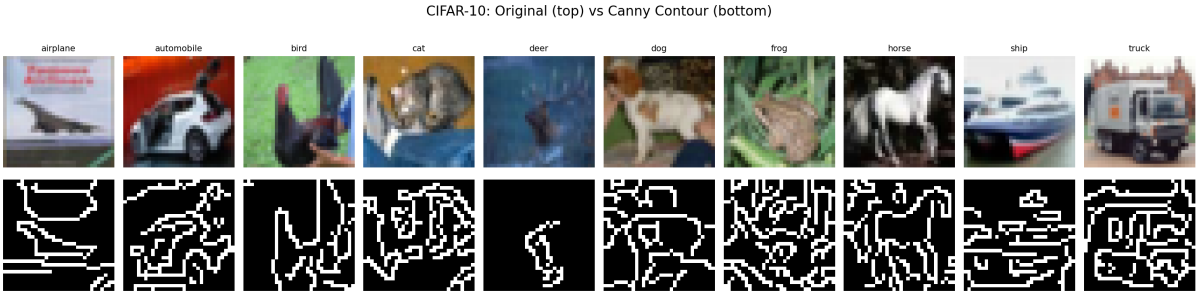

Figure 1: CIFAR-10 samples. Top: originals. Bottom: Canny edge contours.

Figure 2: Overwatch character samples. Top: originals. Bottom: Canny edge contours.

2.2 DINOv2 Setup

We use the `dinov2_vits14` model from the official DINOv2 release. The ViT-S/14 backbone is frozen, and we train only a linear classifier (384-dim $\rightarrow N$ classes) on top of the `[CLS]` token output. Training runs for 100 epochs with the Adam optimizer (lr $= 1 \times 10^{-3}$) and cosine annealing. Batch size is 256 for CIFAR-10 and 64 for Overwatch. Data augmentation (random horizontal flip, color jitter) is applied for Overwatch.

2.3 YOLOv8n-cls Setup

We use the Ultralytics YOLOv8n-cls model, pretrained on ImageNet. All layers are fine-tuned for 30 epochs with the AdamW optimizer (auto-tuned learning rate), image size 224×224. Batch size is 256 for CIFAR-10 and 64 for Overwatch.

2.4 Hardware

All experiments were run on a single NVIDIA RTX 6000 Ada Generation (48 GB) with PyTorch 2.6 and CUDA 12.4.

3 Results: CIFAR-10

3.1 Overall Accuracy

Table 1: Test accuracy (%) on CIFAR-10.

Model	Original	Contour
DINOv2 (ViT-S/14, frozen + linear)	95.89	51.59
YOLOv8n-cls (fine-tuned)	96.02	70.38

Both models perform comparably on original images (∼96%). However, when texture and color are removed, YOLOv8n retains significantly more performance (70.4%) than DINOv2 (51.6%)—a gap of nearly 19 percentage points.

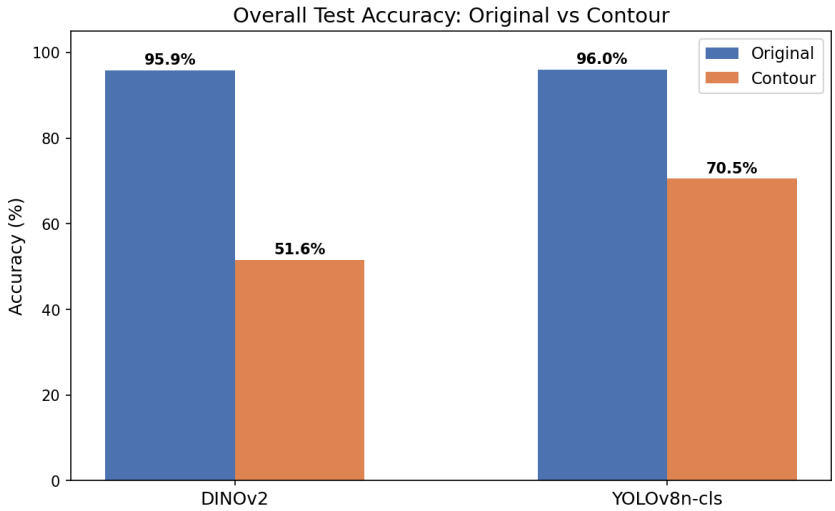

Figure 3: CIFAR-10: Overall accuracy comparison.

3.2 Per-Class Analysis

Figure 4 breaks down accuracy by class. On contour images, YOLO consistently outperforms DINO across all classes. Classes with distinctive shapes (*ship*, *airplane*, *truck*) retain relatively high accuracy for both models, while classes with less distinctive contours (*cat*, *deer*, *dog*) suffer the most.

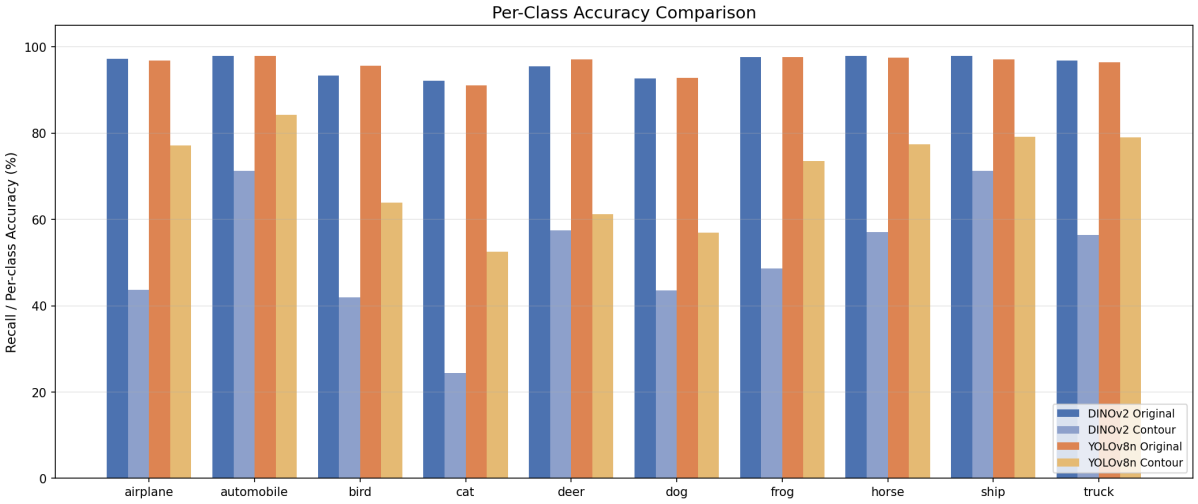

Figure 4: CIFAR-10: Per-class accuracy (recall).

3.3 Confusion Matrices

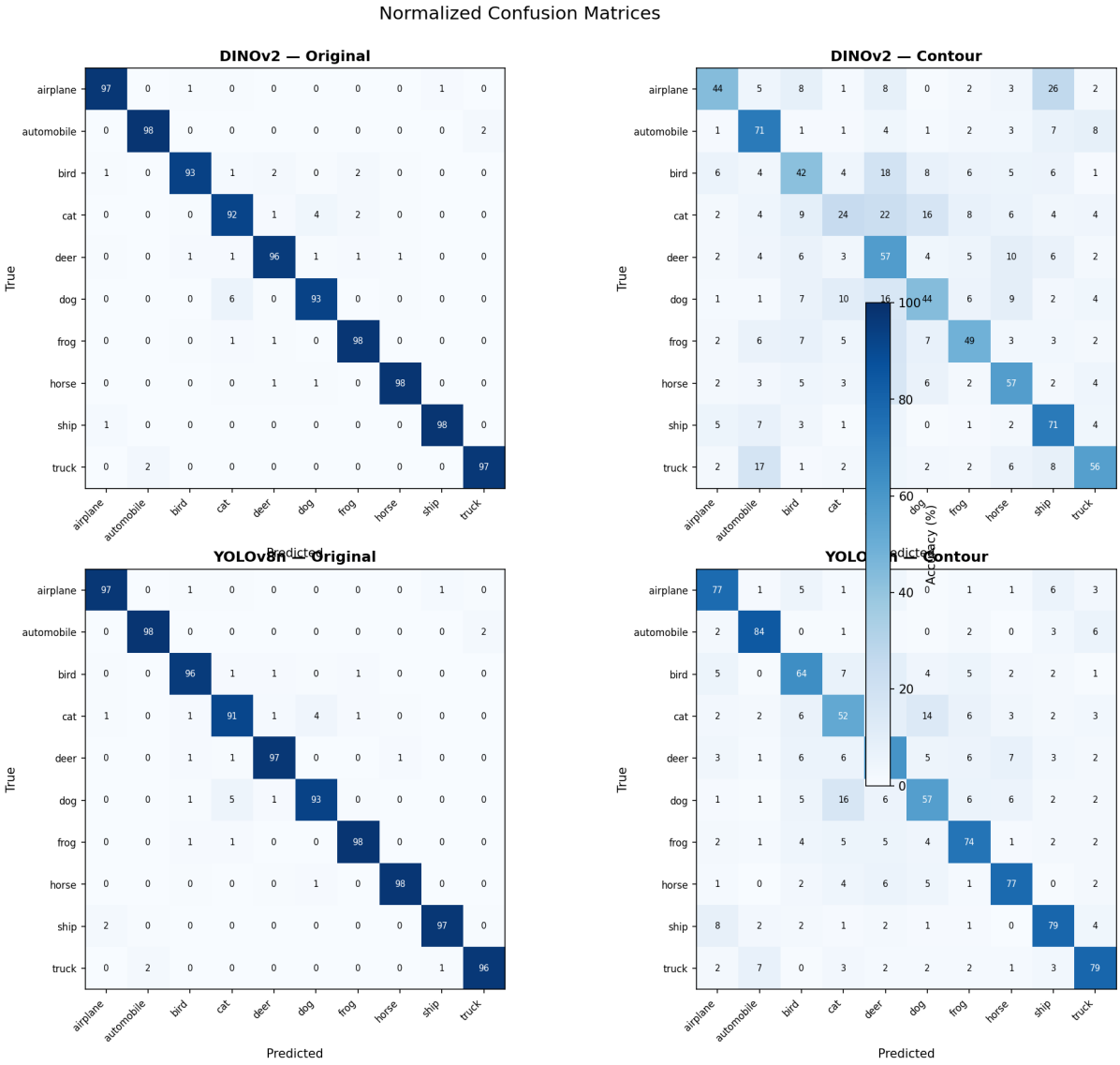

Figure 5: CIFAR-10: Normalized confusion matrices.

3.4 Training Curves

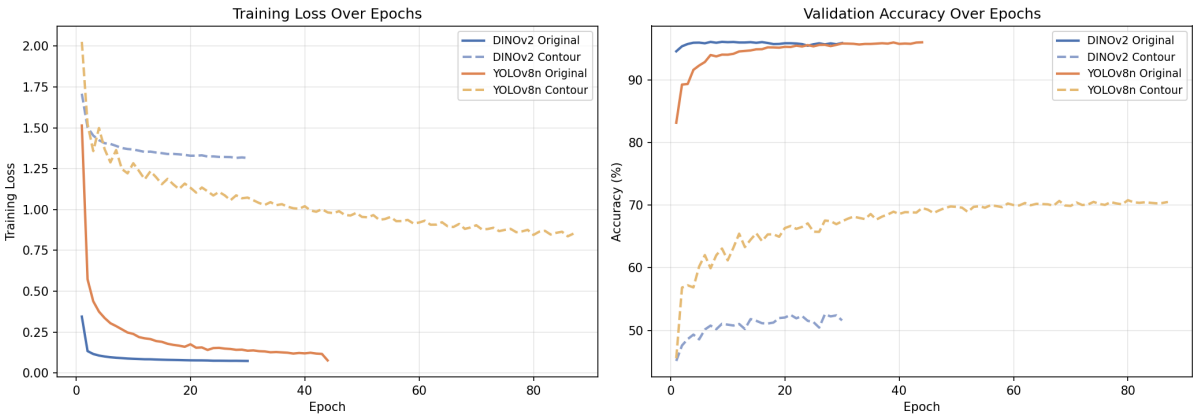

Figure 6: CIFAR-10: Training loss and validation accuracy over epochs.

4 Results: Overwatch Character Recognition

4.1 Overall Accuracy

Table 2: Test accuracy (%) on Overwatch character dataset (14 classes, random baseline = 7.1%).

Model	Original	Contour
DINOv2 (ViT-S/14, frozen + linear)	56.55	13.39
YOLOv8n-cls (fine-tuned)	65.77	38.10

The Overwatch dataset is substantially harder than CIFAR-10: it has more classes (14 vs. 10), far fewer training examples (∼93 per class vs. 5,000), and the characters share similar in-game environments and visual styles. Even on originals, neither model exceeds 66%. On contours, DINO drops to 13.4%—barely above the 7.1% random baseline—while YOLO retains 38.1%.

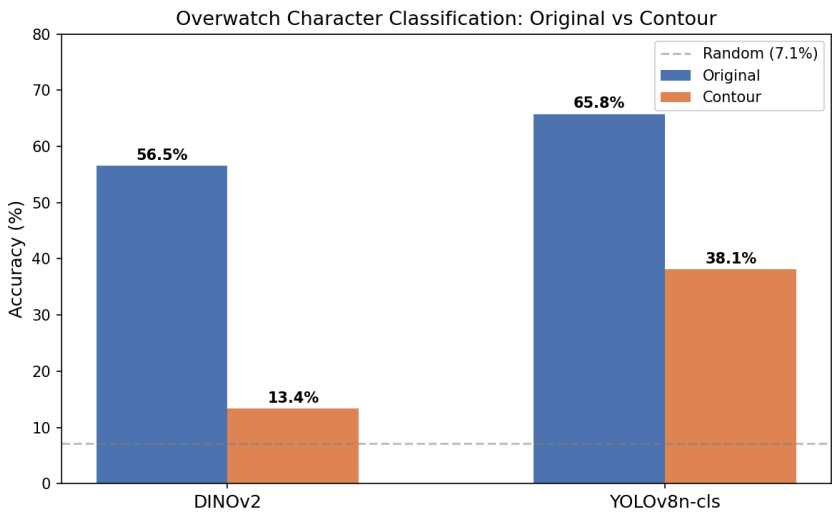

Figure 7: Overwatch: Overall accuracy comparison.

4.2 Per-Class Analysis

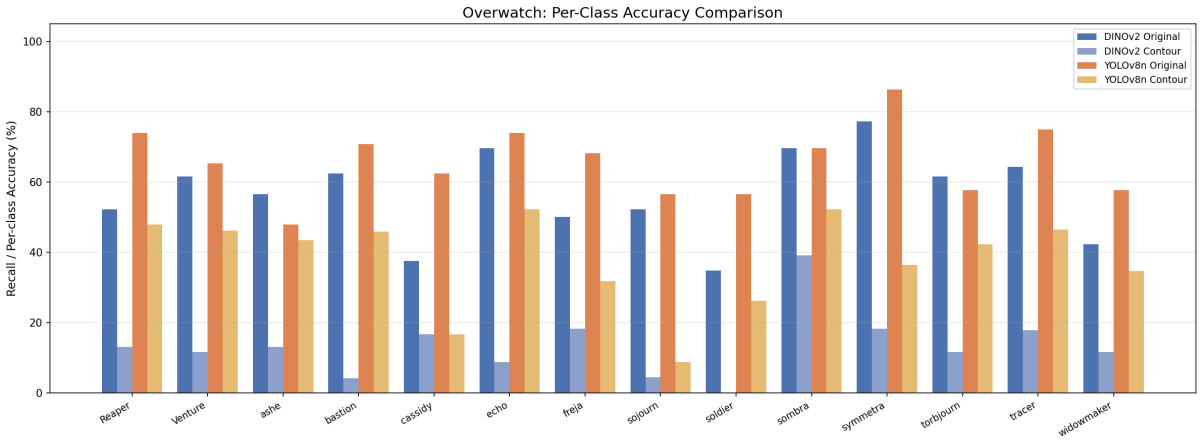

Figure 8: Overwatch: Per-class accuracy (recall).

4.3 Confusion Matrices

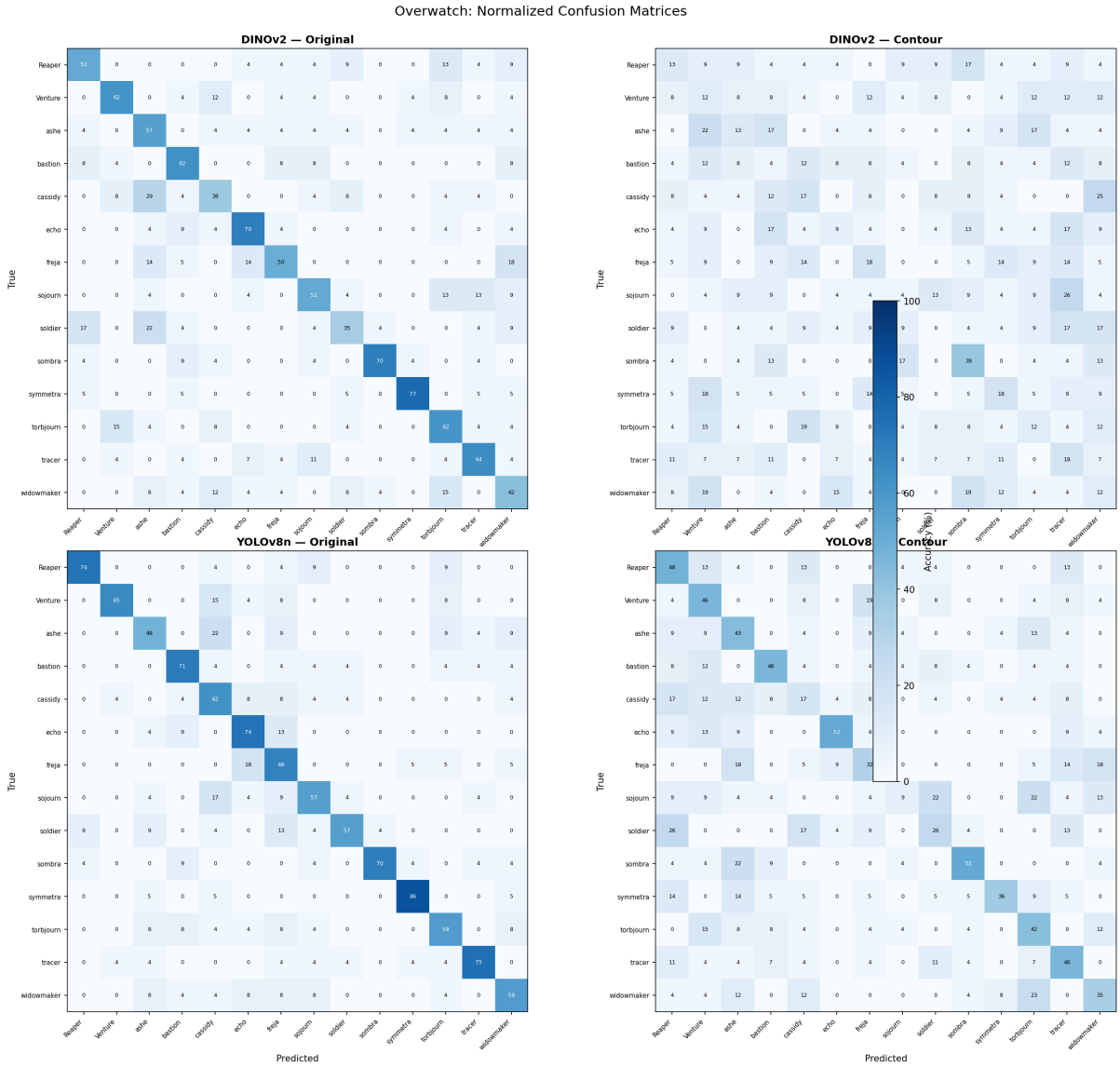

Figure 9: Overwatch: Normalized confusion matrices.

4.4 Training Curves

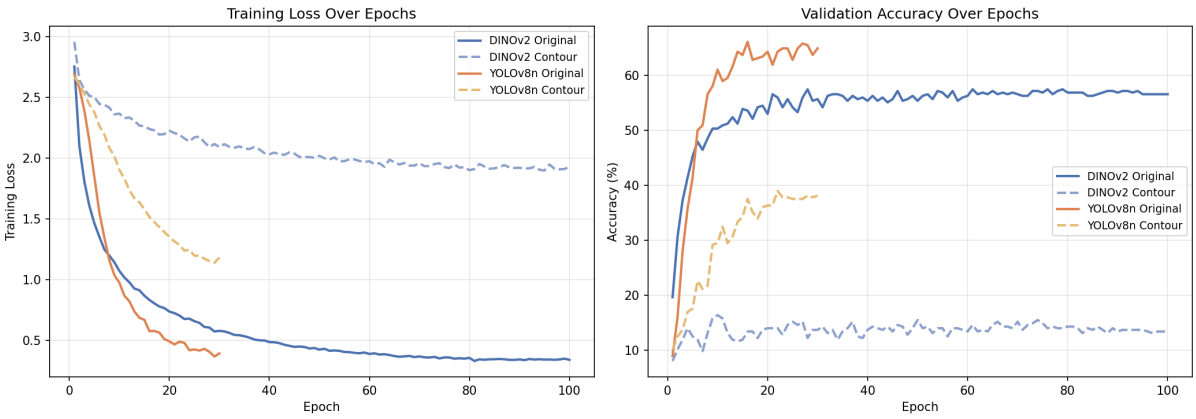

Figure 10: Overwatch: Training loss and validation accuracy over epochs.

5 Discussion

5.1 Cross-Dataset Consistency

The relative ranking of models is consistent across both datasets: YOLOv8n (fine-tuned) outperforms DINOv2 (frozen) on both original and contour images, with the gap widening dramatically on contours. Table 3 summarizes the accuracy drops.

Table 3: Accuracy drop (%) from original to contour.

Model	CIFAR-10 drop	Overwatch drop
DINOv2	−44.3	−43.2
YOLOv8n-cls	−25.6	−27.7

Remarkably, both models lose roughly the same *proportion* of accuracy when switching to contours, regardless of dataset. DINOv2 loses ∼44% (absolute) across both; YOLOv8n loses ∼26–28%.

5.2 Why Does YOLO Outperform DINO on Contours?

The key difference lies in the training paradigm:

- **DINOv2 (frozen backbone + linear probe)**: The features were learned via self-supervised pretraining on natural images rich in texture and color. When these cues are removed, the frozen representations may not encode the remaining shape information in a linearly separable way. The model cannot adapt its internal representations to the contour domain.
- **YOLOv8n (full fine-tuning)**: All layers are updated during training, allowing the network to adapt its low-level and mid-level features specifically for edge/contour patterns. The convolutional inductive bias (local receptive fields, translation equivariance) may also be better suited to processing sparse edge maps.

5.3 Implications

1. **Frozen SSL features are domain-sensitive**: DINOv2's general-purpose representations degrade substantially when the input distribution shifts from natural images to binary edge maps. This holds across both a standard benchmark (CIFAR-10) and a domain-specific dataset (Overwatch).

2. **Shape alone is partially sufficient**: YOLOv8n achieves 70% on CIFAR-10 contours and 38% on Overwatch contours (vs. 7.1% random), demonstrating that Canny edges retain meaningful classification signal.

3. **Adaptation matters more than architecture**: The consistent advantage of YOLO over DINO on contours is primarily due to fine-tuning vs. frozen features, not CNN vs. ViT architecture.

4. **Small datasets amplify the gap**: On Overwatch ($\sim$93 images/class), the limited data makes it even harder for a frozen linear probe to find a good decision boundary, especially in the unfamiliar contour domain.

6 Conclusion

We compared DINOv2 and YOLOv8n-cls on classifying images from two datasets—CIFAR-10 and Overwatch character screenshots—in both original form and as Canny edge contours. Across both datasets, YOLOv8n (fine-tuned) consistently outperforms DINOv2 (frozen) on contours by $\sim$19–25 percentage points, while the two models perform comparably or with a smaller gap on originals. The primary factor is training strategy (fine-tuning vs. frozen backbone), not architecture. Future work could explore fine-tuning DINOv2's backbone, using adapter layers, or testing with larger training sets.

References

[1] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel, "ImageNet-trained CNNs are biased towards textures; increasing shape bias improves accuracy and robustness," in *Proc. ICLR*, 2019.

[2] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," in *Proc. ICCV*, 2021.